



VERIQTA



@veriqta

LEARN. BUILD. DEPLOY. REPEAT.

From
Zero to
Production
Like a Pro

DOCKER

PRODUCTION HANDBOOK

FOUNDATIONS TO PRODUCTION

Build, Run, Debug & Deploy Containers
Like a Production Engineer



MASTER DOCKER
FUNDAMENTALS



BUILD OPTIMIZED
IMAGES



MANAGE DATA
& NETWORKS



SECURE & HARDEN
YOUR CONTAINERS



TROUBLESHOOT LIKE
A PRO



DEPLOY TO
PRODUCTION
WITH CONFIDENCE

Build



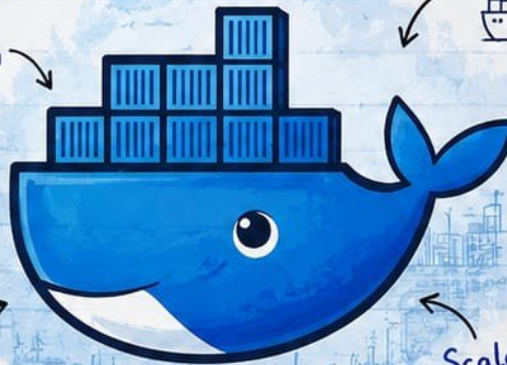
Ship



Run



Scale



WHAT YOU'LL LEARN

- ✓ Docker Architecture
- ✓ Images & Containers
- ✓ Dockerfile Mastery
- ✓ Volumes & Storage
- ✓ Networking Deep Dive
- ✓ Docker Compose
- ✓ Registries & Security
- ✓ Logging & Monitoring
- ✓ Performance Optimization
- ✓ Troubleshooting
- ✓ Real Production Lessons
- ✓ Interview Questions
- ... and much more!



Production First.
Security Always.
Excellence Every Time.

DEVELOP



BUILD



PUSH



DEPLOY



PRODUCTION



FOR DEVOPS ENGINEERS | CLOUD ENGINEERS | SREs | PLATFORM ENGINEERS

REAL-WORLD EXAMPLES
BEST PRACTICES
PRO TIPS
CHEAT SHEETS

A PRACTICAL, HANDS-ON GUIDE FOR ENGINEERS WHO BUILD, SHIP & RUN SYSTEMS THAT MATTER



DOCKER

PRODUCTION HANDBOOK

FOUNDATIONS TO PRODUCTION

Build, Run, Debug & Deploy Containers Like a Production Engineer

“ Docker is not just a tool, it's the foundation of modern software delivery. Master the foundation, and you can conquer production.”



1. WHAT IS DOCKER?

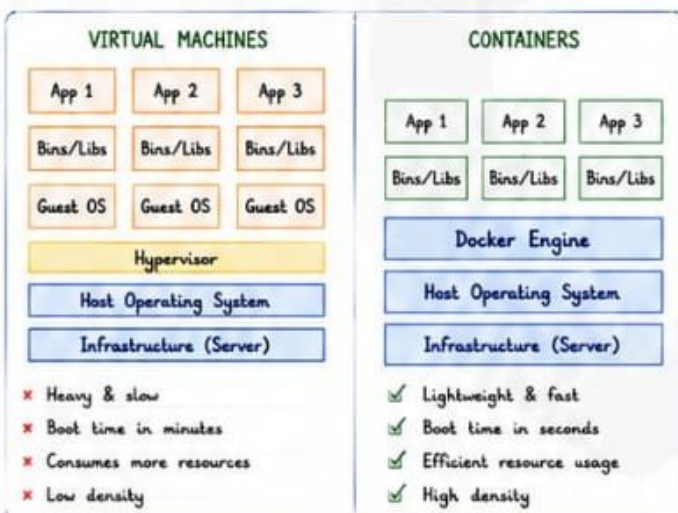
Docker is a platform that allows you to develop, ship and run applications inside containers. Containers package an application with all its dependencies, so it runs consistently across different environments.



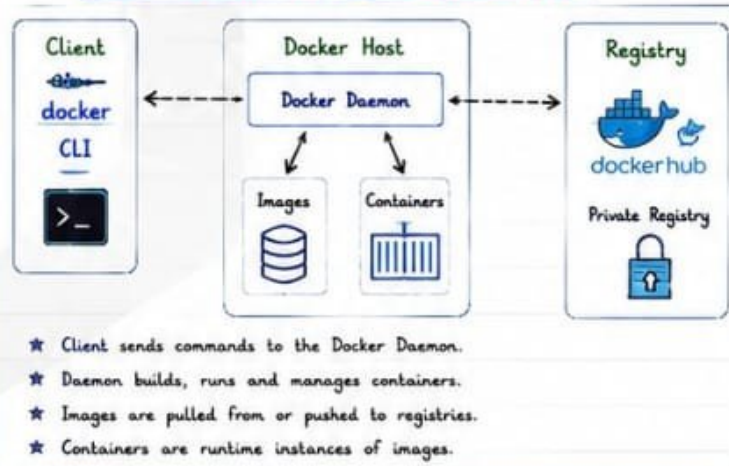
2. WHY DOCKER WAS CREATED

- ✓ Traditional deployments were slow and inconsistent.
- ✓ "Works on my machine" problem.
- ✓ Heavy virtual machines consumed more resources.
- ✓ Containers are lightweight, portable and fast.
- ✓ Built for modern cloud-native, microservices and DevOps workflows.

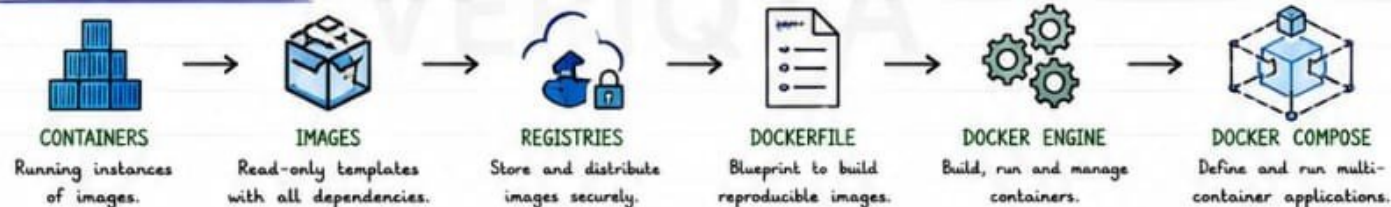
3. VIRTUAL MACHINES VS CONTAINERS



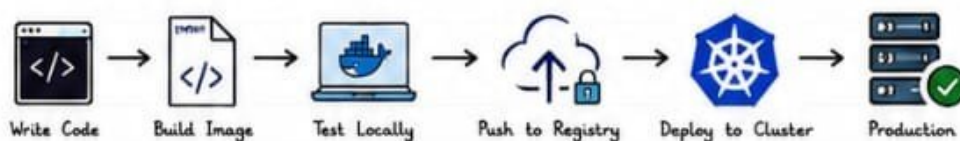
4. DOCKER ARCHITECTURE (HIGH LEVEL)



5. THE DOCKER ECOSYSTEM



6. DOCKER WORKFLOW (FROM CODE TO PRODUCTION)



7. REAL PRODUCTION PIPELINE

- 1 Code committed to Git
- 2 CI/CD pipeline triggered
- 3 Docker image built
- 4 Security scan executed
- 5 Image pushed to registry
- 6 Deployed to Kubernetes
- 7 Running in production 🚀



PRO TIP

Docker is more than a container runtime. It is a complete platform for building, shipping and running modern applications at scale.

REMEMBER

Containers are isolated.
Images are immutable.
Infrastructure is disposable.
Automation is repeatable.



Master the foundation in **Volume 1**, and you are ready to build, run and deploy anything anywhere.



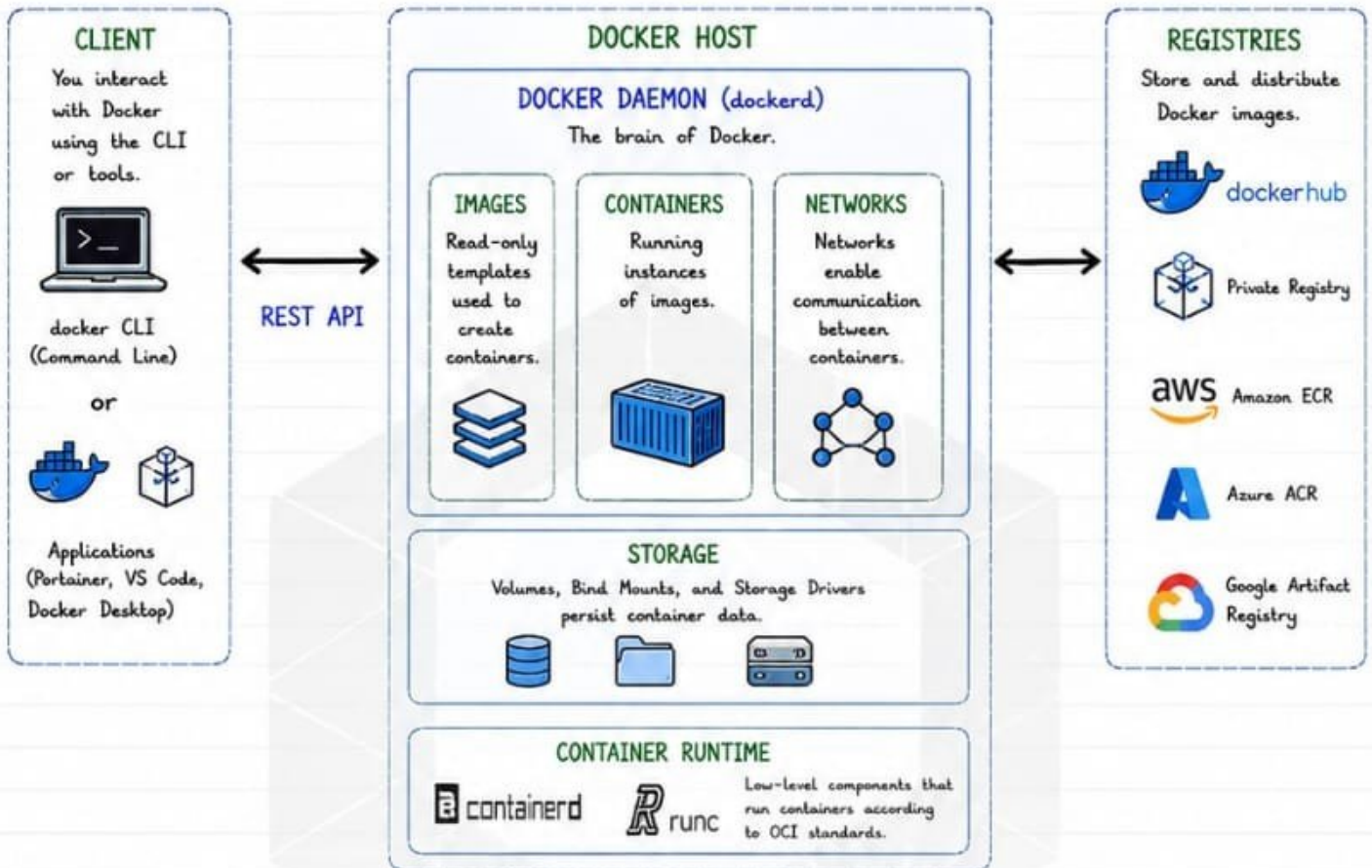


DOCKER ARCHITECTURE DEEP DIVE

★ Understand the Engine Behind Every Container ★



Docker uses a client-server architecture. The Docker Client talks to the Docker Daemon, which builds, runs and manages Docker objects.



COMPONENTS EXPLAINED

- **Docker Client** - Sends commands to the Docker Daemon.
- **Docker Daemon** - Builds, runs, distributes and manages Docker objects.
- **Images** - Immutable templates used to create containers.
- **Containers** - Isolated runtime instances of images.
- **Storage** - Manages persistent data for containers.
- **Networks** - Enables communication between containers and the outside world.
- **Registries** - Stores and distributes container images.
- **REST API** - Communication channel between Client and Daemon.
- **containerd** - Manages container lifecycle and images.
- **runc** - Creates and runs containers using OCI standards.
- **OCI (Open Container Initiative)** - Industry standard for container formats and runtimes.
- **Docker Host** - The machine (physical or virtual) running Docker Engine.

DOCKER ARCHITECTURE FLOW

1. You run a command in the Docker Client. (`docker run nginx`)
2. Client sends the request to the Docker Daemon.
3. Daemon pulls image from Registry (if not available).
4. Daemon creates a container using `containerd` + `runc`.
5. Container runs, attached to networks and storage.



KEY TAKEAWAYS



- ✓ Docker uses a Client-Server model.
- ✓ Daemon is the core that does the heavy lifting.
- ✓ Images are templates; Containers are runtime.
- ✓ Storage and Networks are essential for production workloads.
- ✓ Registries are the source of truth for images.
- ✓ `containerd` and `runc` ensure containers run securely and efficiently.

WHY THIS MATTERS IN PRODUCTION



- ★ High availability depends on the daemon and host stability.
- ★ Networking and storage design impact performance and reliability.
- ★ Images from trusted registries ensure security and consistency.
- ★ Understanding this architecture helps you troubleshoot faster and design better systems.



Master the architecture today, build **unbreakable systems** tomorrow.





INSTALLING DOCKER LIKE A PRODUCTION ENGINEER

★ Choose Your OS. Install Correctly. Verify. Secure. ★



Always install the latest stable version from official sources. Never from random scripts.
Official Docs: docs.docker.com/engine/install

1. UBUNTU / DEBIAN

1. Update packages

```
sudo apt update && sudo apt upgrade -y
```

2. Install prerequisites

```
sudo apt install -y ca-certificates \
curl gnupg lsb-release
```

3. Add Docker's official GPG key

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor /dev/null
sudo mv docker.gpg /usr/share/keyrings/
echo "deb [arch=amd64] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

5. Install Docker Engine

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

2. RHEL / CENTOS / ROCKY / ALMALINUX

1. Install prerequisites

```
sudo yum install -y yum-utils
```

2. Add Docker repo

```
sudo yum-config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

3. Install Docker Engine

```
sudo yum install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

4. Start & enable Docker

```
sudo systemctl enable --now docker
```

3. AMAZON LINUX 2

1. Update packages

```
sudo yum update -y
```

2. Install Docker

```
sudo amazon-linux-extras install docker -y
```

3. Start & enable

```
sudo systemctl enable --now docker
```

4. DOCKER DESKTOP

For Mac / Windows / Linux users who need a full local Docker experience.



- ☒ Download Docker Desktop from official site.
- ☒ Install and follow wizard.
- ☒ Enable WSL 2 (Windows).
- ☒ Ensure virtualization is on.
- ☒ Sign in to Docker Hub (optional).

5. VERIFY INSTALLATION 🔍

Check Docker version

```
docker --version
docker version
```

You should see a message that Docker is installed and working! ✓

Run hello-world

```
docker run hello-world
```

6. DOCKER SERVICE MANAGEMENT (PRODUCTION ESSENTIAL) ⚙️

Start

```
sudo systemctl start docker
```

Stop

```
sudo systemctl stop docker
```

Restart

```
sudo systemctl restart docker
```

Enable on boot

```
sudo systemctl enable docker
```

Check status

```
sudo systemctl status docker
```

Disable on boot

```
sudo systemctl disable docker
```

Note:

Always ensure Docker starts on boot for production servers. ✓

7. COMMON INSTALLATION PROBLEMS ⚠️

PROBLEM	CAUSE	SOLUTION
Cannot connect to Docker Daemon	Docker service not running	<pre>sudo systemctl start docker sudo systemctl enable docker</pre>
Permission denied (docker.sock)	User not in docker group	<pre>sudo usermod -aG docker \$USER newgrp docker</pre>
Repository not found	Wrong repo or OS version	Check OS version Re-add correct Docker repo
Package conflict	Old docker packages installed	<pre>sudo yum remove docker \ docker-client docker-client-latest \ docker-common docker-latest \ docker-latest-logrotate docker-logrotate</pre>
SELinux issues	SELinux blocking Docker	<pre>setenforce 0</pre> (temporary) Or configure policies

8. BEST PRACTICES ★

- ☒ Always install from official repos.
- ☒ Keep Docker Engine updated regularly.
- ☒ Use `non-root` user when possible.
- ☒ Enable Docker to start on boot.
- ☒ Use Docker Compose for multi-container apps.
- ☒ Use dedicated volumes for persistent data.
- ☒ Scan images for vulnerabilities.
- ☒ Use least privilege principle.
- ☒ Back up important data & volumes.
- ☒ Monitor Docker service & resources.



Treat Docker as part of your infrastructure, not a toy.
Secure it. Monitor it. Maintain it.

9. QUICK COMMANDS CHEAT SHEET ⚡

<code>docker --version</code>	→ Check version
<code>docker run hello-world</code>	→ Test installation
<code>docker ps</code>	→ List running containers
<code>docker ps -a</code>	→ List all containers
<code>docker images</code>	→ List images
<code>docker info</code>	→ System-wide info
<code>docker system df</code>	→ Disk usage
<code>docker logout</code>	→ Logout from registry



A strong installation today prevents 80% of problems tomorrow. Install right. Run right. Scale right.





DOCKER IMAGES

☆ The Blueprint of Your Containers ☆



A Docker image is a read-only template with all the instructions and dependencies needed to create containers.

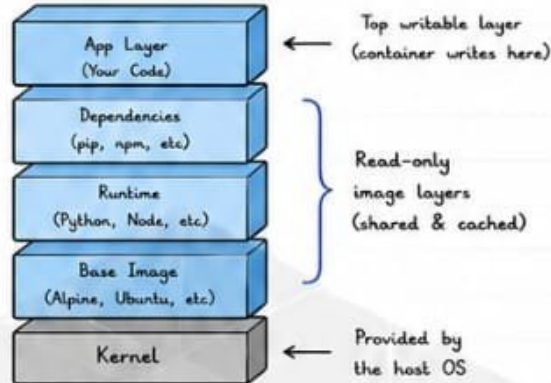
1. WHAT IS A DOCKER IMAGE?

A Docker image is a lightweight, standalone, executable package that includes everything needed to run a piece of software:

- Code
- Runtime
- Libraries
- Environment Variables
- Configuration Files
- Dependencies



2. IMAGE LAYERS (UNION FILE SYSTEM)



3. KEY CHARACTERISTICS

- ✓ Immutable (read-only)
- ✓ Shareable
- ✓ Versioned (via tags)
- ✓ Layered for efficiency
- ✓ Lightweight
- ✓ Secure
- ✓ Portable across environments
- ✓ Built from Dockerfile



4. OFFICIAL VS CUSTOM IMAGES

OFFICIAL IMAGES	CUSTOM IMAGES
Built & maintained by Docker	Built by you or your team
Trusted & secure	You control content & security
Well-documented	Tailored to your needs
Example: nginx, redis, mysql	Example: myapp:v1, api:prod

5. IMAGE TAGS



- **latest** → Default tag (not always latest release)
- Use explicit tags in production.
- Avoid **:latest** in production.

6. PULL VS BUILD

PULL (FROM REGISTRY)



`docker pull nginx:1.25`

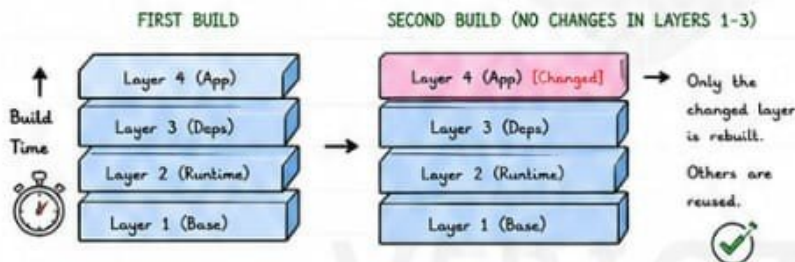
BUILD (FROM DOCKERFILE)



`docker build -t myapp:1.0`

7. IMAGE LAYER CACHING (WHY IT MATTERS)

Docker reuses unchanged layers → faster builds, smaller storage, efficient deployments.



8. COMMON IMAGE COMMANDS

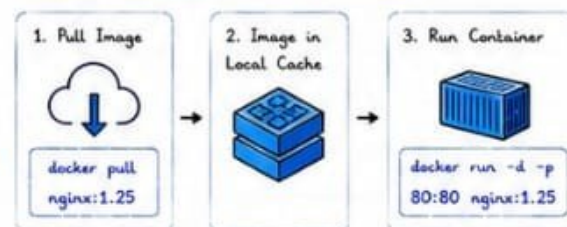
- `docker images` → List all images
- `docker pull <image>:<tag>` → Pull image
- `docker rmi <image>:<tag>` → Remove image
- `docker inspect <image>` → Inspect image details
- `docker history <image>` → Show image layers
- `docker tag <img> <new:tag>` → Tag image
- `docker save <img> -o image.tar` → Save to file
- `docker load -i image.tar2` → Load from file
- `docker image prune -a` → Remove unused images

9. BEST PRACTICES

- ✓ Use official images as base.
- ✓ Pin exact tags (no :latest).
- ✓ Keep images small (use Alpine or slim variants).
- ✓ Use multi-stage builds.
- ✓ Remove unnecessary packages.
- ✓ Scan images for vulnerabilities.
- ✓ Don't store secrets in images.
- ✓ Rebuild and push regularly.



10. EXAMPLE: FROM PULL TO RUN



PRO TIP



Images are the foundation of everything in Docker.

Build them clean.
Keep them lean.
Secure them always.



A clean, well-built image today = reliable, scalable, secure containers tomorrow.



★ The Running Instance of an Image ★



If images are blueprints, containers are the running buildings. Containers are isolated, portable, and lightweight.

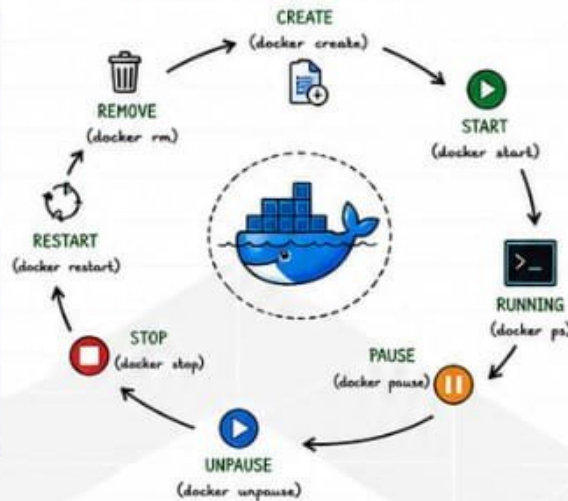
1. WHAT IS A CONTAINER?

A container is a runnable instance of an image. It has its own filesystem, process space, network and can start, stop, move or delete independently.



- ✓ Multiple containers can run from the same image.
- ✓ Each container has a unique writable layer.

2. CONTAINER LIFECYCLE



3. CONTAINER STATES

- CREATED**
Container is created but not started.
- RUNNING**
Container is running and active.
- PAUSED**
All processes are frozen.
- STOPPED**
Container is stopped but still exists.
- REMOVED**
Container is deleted (cannot be recovered).

4. COMMON CONTAINER COMMANDS

TASK	COMMAND	EXAMPLE	DESCRIPTION
Create a container	<code>docker create <image></code>	<code>docker create nginx:1.25</code>	Creates container but does not start.
Run a container	<code>docker run [options] <image></code>	<code>docker run -d -p 80:80 nginx</code>	Creates and starts the container.
List running containers	<code>docker ps</code>	<code>docker ps</code>	Shows only running containers.
List all containers	<code>docker ps -a</code>	<code>docker ps -a</code>	Shows all containers (running + stopped).
Stop a container	<code>docker stop <container_id></code>	<code>docker stop a1b2c3d4e5f6</code>	Stops the running container.
Start a container	<code>docker start <container_id></code>	<code>docker start a1b2c3d4e5f6</code>	Starts a stopped container.
Restart a container	<code>docker restart <container_id></code>	<code>docker restart a1b2c3d4e5f6</code>	Restarts the container.
Pause a container	<code>docker pause <container_id></code>	<code>docker pause a1b2c3d4e5f6</code>	Pauses all processes inside.
Unpause a container	<code>docker unpause <container_id></code>	<code>docker unpause a1b2c3d4e5f6</code>	Resumes processes.
Remove a container	<code>docker rm <container_id></code>	<code>docker rm a1b2c3d4e5f6</code>	Removes stopped container.
Force remove	<code>docker rm -f <container_id></code>	<code>docker rm -f a1b2c3d4e5f6</code>	Force removes container.
Execute command	<code>docker exec -it <container> <cmd></code>	<code>docker exec -it nginx bash</code>	Runs command inside container.
View logs	<code>docker logs <container></code>	<code>docker logs nginx</code>	Shows container logs.
Inspect container	<code>docker inspect <container></code>	<code>docker inspect nginx</code>	Displays detailed info (JSON).

5. MOST USED DOCKER RUN OPTIONS

- `-d` Run in detached mode (background)
- `-it` Interactive terminal
- `-p` Publish port (host:container)
- `-v` Mount volume
- `--name` Assign container name
- `--rm` Remove container when it stops
- `-e` Set environment variable
- `--restart` Restart policy (always, unless-stopped, etc.)
- `--network` Connect to a network
- `--user` Run as specific user

EXAMPLE:

```
docker run -d \
  --name web1 \
  -p 8080:80 \
  -v /data/web/usr/share/nginx/html \
  -e APP_ENV=production \
  --restart unless-stopped \
  nginx:1.25
```



6. REAL WORLD EXAMPLE



- ★ The image is static.
- The container is dynamic.
- You can stop it, start it, move it or remove it.

PRO TIP

Think of containers as cattle, not pets. They are replaceable, disposable and automated.

REMEMBER

- Images don't run.
- Containers run.
- Containers can be created, started, stopped, and removed quickly.

NEXT PAGE PREVIEW

We will learn how to create optimized images using Dockerfile.



Master containers today, and you master the foundation of modern infrastructure.





DOCKERFILE MASTERCLASS

★ The Instructions That Build Production-Ready Images ★



A Dockerfile is a blueprint. Each instruction creates a layer in the image. Write it clean, secure, efficient and reproducible.

1. WHAT IS A DOCKERFILE?

A Dockerfile is a text file that contains a set of instructions to build a Docker image. Each instruction creates a new layer.



build



2. DOCKERFILE INSTRUCTIONS OVERVIEW

INSTRUCTION	PURPOSE	EXAMPLE
FROM	Base image for the build	FROM alpine:3.19
RUN	Execute commands in image	RUN apk add --no-cache nginx
COPY	Copy files from host to image	COPY . /app
ADD	Copy & extract tar files / URLs	ADD app.tar.gz /app
CMD	Default command to run	CMD ["nginx", "-g", "daemon off;"]
ENTRYPOINT	Executable that always runs	ENTRYPOINT ["docker-entrypoint.sh"]
WORKDIR	Set working directory	WORKDIR /app
ENV	Set environment variable	ENV NODE_ENV=production
EXPOSE	Inform Docker of port	EXPOSE 8080
LABEL	Add metadata to image	LABEL maintainer="veriqta"
USER	Run as a specific user	USER nginx
HEALTHCHECK	Check container health	HEALTHCHECK CMD curl -f localhost

3. ANNOTATED DOCKERFILE EXAMPLE

```

1 FROM node:18-alpine           # Base image
2
3 WORKDIR /app                  # Set working directory
4
5 COPY package*.json ./         # Copy dependency files
6 RUN npm ci --only=production  # Install dependencies
7
8 COPY . .                      # Copy application code
9
10 ENV NODE_ENV=production       # Set environment
11 EXPOSE 8080                   # Expose application port
12
13 USER node                     # Run as non-root user
14
15
16 HEALTHCHECK --interval=30s \
46   CMD curl -f http://localhost:8080/ || exit 1 # Health check

```

EXPLANATION

- Start from a lightweight base image (Alpine).
- Set working directory for all following commands.
- Install production dependencies first (better caching).
- Copy the rest of the application.
- Set environment variables.
- Expose the port your app will listen on.
- Run as non-root for security.
- Provide a health check for orchestration systems.

4. BEST PRACTICES FOR WRITING DOCKERFILES



1. KEEP IT SMALL

Use official lightweight base images like alpine.



2. ORDER MATTERS

Put stable instructions first to leverage cache.



3. CLEAN UP

Remove unnecessary files and package manager caches.



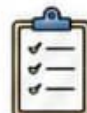
4. SECURITY FIRST

Run as non-root user and avoid secrets in images.



5. USE .DOCKERIGNORE

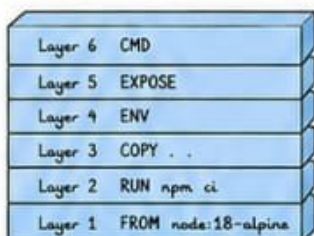
Exclude unnecessary files from build context.



6. TEST YOUR BUILD

Always test locally before pushing to production.

5. LAYERS VISUALIZATION



Top
↑
Base

NOTES

- Each instruction adds a new layer.
- Layers are cached and reused when possible.
- Smaller layers = faster builds.
- Changes in lower layers invalidate upper layers.

6. COMMON MISTAKES TO AVOID

- ✗ Using latest tag in production.
- ✗ Installing unnecessary packages.
- ✗ Running as root user.
- ✗ Copying everything (without .dockerignore).
- ✗ Not cleaning package manager caches.
- ✗ Storing secrets in Dockerfile.
- ✗ Large base images.
- ✗ Not using multi-stage builds.

PRO TIP



A good Dockerfile is not about making it work, it's about making it efficient, secure, maintainable and production-ready.



A great Dockerfile today = faster builds, smaller images, secure containers, happy users tomorrow.





BUILDING PRODUCTION IMAGES

★ From Dev Machine to Production Grade ★



A good image is small, secure, fast to pull, and predictable.
Build smart today, deploy confidently tomorrow.

1. DOCKER BUILD BASICS

`docker build [OPTIONS] -t <image>:<tag> <path>`

- `-t, --tag` Name and tag the image.
- `<path>` Path to the build context.
- `.` Current directory (most common).
- Build sends context to Docker daemon.

EXAMPLE

`docker build -t myapp:1.0 .`



PRO TIP

Always build from the root of your project.
Keep context small and clean.



2. BUILD CONTEXT

The build context is the set of files Docker can see during build.

BUILD CONTEXT (.)

```
myapp/
├── Dockerfile
├── app/
│   └── main.py
├── requirements.txt
├── config/
│   └── settings.yaml
├── .dockerignore
└── README.md
```

NOTE:

Only files inside the context can be copied into the image.

3. .DOCKERIGNORE (DON'T SEND UNNECESSARY FILES)

EXAMPLE .DOCKERIGNORE

```
.git
__pycache__/
node_modules/
*.log
.env
*.pem
docker-compose.yml
README.md
venv/
.idea/
```

WHY IT MATTERS

- ✓ Faster builds
- ✓ Smaller context
- ✓ Improved security
- ✓ Less data transferred
- ✓ Lower build cache invalidation

BAD VS GOOD CONTEXT SIZE

WITHOUT .DOCKERIGNORE



512 MB

WITH .DOCKERIGNORE



12 MB

4. BUILD CACHE (YOUR BEST FRIEND)

Docker caches each layer. If nothing changes, Docker reuses the layer. This makes builds fast and efficient.

FIRST BUILD



Takes longer

SECOND BUILD (NO CHANGES)



Much faster



TIP:

Order instructions from least likely to change → most likely to change. This maximizes cache hits.

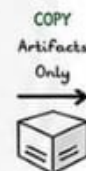
5. MULTI-STAGE BUILDS (PRODUCTION ESSENTIAL)

Use multi-stage builds to keep final images small, secure and clean.

BUILD STAGE (Builder Image)



Compiles code, installs build deps, creates artifacts



COPY Artifacts Only

FINAL STAGE (Runtime Image)



Contains only runtime deps + application

BENEFITS

- ✓ Smaller final image
- ✓ No build tools in production
- ✓ Better security
- ✓ Faster pull & deploy

EXAMPLE USE CASES

- Node.js apps
- Python apps
- Java apps
- Go apps
- .NET apps

6. IMAGE OPTIMIZATION TECHNIQUES

- ✓ Use minimal base images (Alpine, Distroless, Slim).
- ✓ Use multi-stage builds.
- ✓ Clean up package manager caches.
- ✓ Remove unnecessary files and docs.
- ✓ Use .dockerignore.
- ✓ Avoid installing build tools in final image.
- ✓ Combine RUN commands to reduce layers.
- ✓ Pin versions for reproducibility.

BAD

- ✗ FROM ubuntu:22.04
- ✗ RUN apt update
- ✗ RUN apt install -y curl
- ✗ RUN apt install -y git
- ✗ RUN apt install -y vim
- ✗ RUN rm -rf /var/lib/apt/lists/*

GOOD

- ✓ FROM ubuntu:22.04
- ✓ RUN apt update && \
- apt install -y --no-install-recommends \
- curl git vim && \
- rm -rf /var/lib/apt/lists/*

SIZE COMPARISON (EXAMPLE)

BAD IMAGE 420 MB

GOOD IMAGE 76 MB



REMEMBER

Your image runs in production. Treat size, speed and security as features.



7. SECURITY BEST PRACTICES

- ✓ Use official and trusted base images.
- ✓ Scan images for vulnerabilities.
- ✓ Run as non-root user.
- ✓ Don't store secrets in images.
- ✓ Use multi-stage builds.
- ✓ Keep packages updated.
- ✓ Follow least privilege principle.

AVOID

- ✗ Latest tag in production
- ✗ Root user
- ✗ Unnecessary packages
- ✗ Secrets in ENV or files
- ✗ Copying everything (*)
- ✗ Exposing unnecessary ports



8. REAL WORLD DOCKER BUILD WORKFLOW



Code



Dockerfile



Build Image



Tag Image



Push Image



Deploy

COMMAND SNAPSHOT

```
docker build -t myapp:1.0 .
docker tag myapp:1.0 registry.com/myapp:1.0
docker push registry.com/myapp:1.0
```



PRO TIP

Automate builds with CI/CD. Use tests, scans and policies before pushing.



A well-built image is the foundation of a stable, secure and scalable system.





DOCKER VOLUMES

★ Persist Data, Keep Containers Stateless ★



Containers are ephemeral. Data should be persistent.
Volumes are the persistent storage layer for containers.

1. WHY DO WE NEED VOLUMES?

- Containers are disposable. Data in a container is lost when the container is removed.
- Volumes persist data beyond the container lifecycle.
- Share data between containers.
- Back up and restore data easily.
- Better performance and storage management.

WITHOUT VOLUME

Container deleted
= Data lost ❌



Data Gone!

WITH VOLUME

Container deleted
= Data safe ✅



Data Persists!

2. TYPES OF VOLUMES

TYPE	DESCRIPTION	BEST FOR	LIFESPAN
Named Volumes	Managed by Docker in /var/lib/docker/volumes	Databases, apps, production data	Until manually deleted
Bind Mounts	Map host directory or file to container	Dev, logs, config files	Until host file deleted
tmpfs Mounts	Stored in memory, not on disk	Sensitive data, high performance	Until container stops
Volume Plugins	Use external storage drivers (cloud, NFS, etc)	Enterprise, cloud storage	Depends on driver

3. NAMED VOLUMES

Managed by Docker. Stored in Docker host.

COMMANDS

```
docker volume create mydata # create volume
docker volume ls # list volumes
docker volume inspect mydata # inspect volume
docker volume rm mydata # remove volume
```

USAGE EXAMPLE

```
docker run -d --name db -v mydata:/var/lib/mysql mysql:8.0
```

↑ Volume on host ↑ Mount point in container

Docker Host



/var/lib/docker/volumes/mydata/_data

Mounted

Container



/var/lib/mysql

4. BIND MOUNTS

Map a file or directory from host to container.

COMMANDS

```
docker run -v /host/path:/container/path image
docker run -v /host/file:/container/file image:tag
```

USAGE EXAMPLE

```
docker run -d --name web -v /app/data:/var/www/html nginx
```

↑ Host path ↑ Container path

Docker Host



/app/data (your files)

Mounted

Container



/var/www/html

NOTE

Changes on host immediately reflect inside the container (and vice versa).

5. TMPFS MOUNTS (IN-MEMORY)

Stored in memory. Fast and secure. Lost on restart.

EXAMPLE

```
docker run --tmpfs /run --tmpfs /tmp nginx
```

RAM



In-Memory

Mounted

Container



/run, /tmp

6. VOLUME COMPARISON

FEATURE	NAMED VOLUME	BIND MOUNT	TMPFS MOUNT
Managed by Docker	Yes	No	Yes
Location	Docker Host	Host File System	Memory (RAM)
Performance	High	Medium	Very High
Persistence	Yes	Yes	No
Best For	Production Data	Dev & Configs	Sensitive Data
Survives Container Deletion	Yes	Yes	No

7. DATA BACKUP STRATEGIES

NAMED VOLUME BACKUP

```
docker run --rm -v mydata:/data -v $(pwd):/backup \
alpine tar cxf /backup/mydata_backup.tar.gz -C /data
```

RESTORE NAMED VOLUME

```
docker run --rm -v mydata:/data -v $(pwd):/backup \
alpine tar xzf /backup/mydata_backup.tar.gz -C /data
```

BIND MOUNT BACKUP

```
tar cxf backup.tar.gz /host/path
```

RESTORE BIND MOUNT

```
tar xzf backup.tar.gz -C /host/path
```

8. REAL WORLD SCENARIOS

- Databases (MySQL, Postgres)
Use named volumes for data persistence.
- Logs & Configs
Use bind mounts for easy access and updates.
- Sensitive Data (Tokens, Keys)
Use tmpfs mounts to keep in memory only.
- Cloud Storage
Use volume plugins (EBS, NFS, Azure Disk, etc).

BEST PRACTICES

- Use named volumes for production data.
- Use bind mounts only when necessary.
- Never store sensitive data in bind mounts.
- Back up volumes regularly.
- Use proper permissions on mount points.
- Document volume usage in your setup.
- Monitor disk usage with: `docker system df`

9. COMMON MISTAKES

- Storing database data in bind mounts.
- Mounting entire host directories.
- Not backing up important volumes.
- Using tmpfs for large data.
- Forgetting to set proper permissions.
- Deleting volumes without backup.

PRO TIP

Keep containers stateless. Store data in volumes. Your future self will thank you!



Volumes give your containers memory. Strategy makes them production-ready.





DOCKER NETWORKING

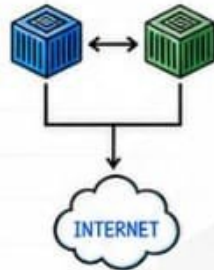
★ Connect Containers, The Right Way ★



Networking is what makes containers communicate.
Docker networking is simple, powerful and flexible.

1. WHY NETWORKING MATTERS

- ✓ Containers need to talk to:
 - Other containers
 - The host system
 - External services (DB, APIs, Internet)
- ✓ Docker provides isolated and secure networks out of the box.
- ✓ You control how containers connect.



2. DOCKER NETWORK DRIVERS

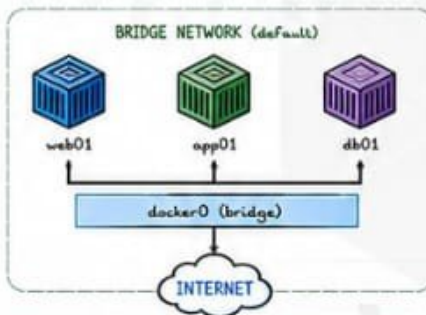
DRIVER	SCOPE	DESCRIPTION	USE CASE
bridge	Single Host	Default network for containers on one host	General purpose (standalone)
host	Single Host	Container shares the host network	High performance apps
none	Single Host	No networking (configured by user)	Highly isolated containers
overlay	Multi-Host	Connect containers across multiple hosts	Swarm mode (Multi-host)
macvlan	Single Host	Container gets its own MAC on the network	Legacy apps, network control

3. DEFAULT BRIDGE NETWORK

When you run a container, Docker connects it to the "bridge" network by default.

Containers can:

- ✓ Talk to each other by container name.
- ✓ Access the internet.
- ✓ Be isolated from other Docker hosts.



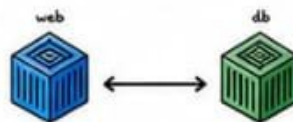
4. CONTAINER COMMUNICATION

Containers on the same network can communicate using container name.

EXAMPLE

```
$ docker run -d --name db mysql:8.0
```

```
$ docker run -d --name web --link db:db nginx
```



web connects to database using hostname: db

5. NETWORK COMMANDS CHEAT SHEET

COMMAND	DESCRIPTION
<code>docker network ls</code>	List all networks
<code>docker network inspect <net></code>	Inspect a network
<code>docker network create <name></code>	Create a new network
<code>docker network rm <name></code>	Remove a network
<code>docker network connect <net> <container></code>	Connect container to network
<code>docker network disconnect <net> <container></code>	Disconnect container from network
<code>docker network prune</code>	Remove unused networks

6. CUSTOM BRIDGE NETWORK (BEST PRACTICE)

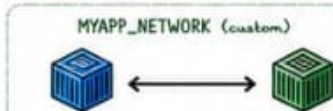
Create custom networks to isolate your applications.

EXAMPLE

```
$ docker network create myapp_network
```

```
$ docker run -d --name db --network myapp_network mysql:8.0
```

```
$ docker run -d --name web --network myapp_network nginx
```

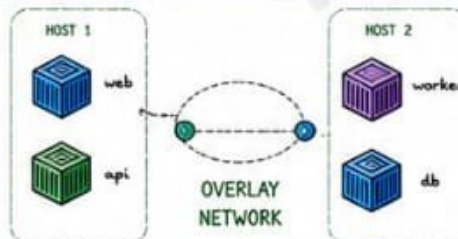


Benefits:

- ✓ Better isolation
- ✓ DNS name resolution
- ✓ Easier management

7. OVERLAY NETWORK (MULTI-HOST)

Used in Docker Swarm to connect containers across hosts.



Allows containers on different hosts to communicate as if they are on the same network.

8. NETWORK TIPS

- ✓ Use custom bridge networks for each project.
- ✓ Avoid `--link` (deprecated). Docker DNS is better.
- ✓ Use overlay networks for multi-host setups.
- ✓ Use host network for performance-critical apps.
- ✓ Keep your network design simple and clear.

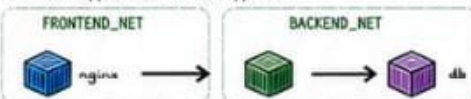
QUICK NOTE

Docker provides secure, isolated networks so your containers can communicate without exposing unnecessary ports.



9. REAL WORLD SCENARIO

A 3-tier application (Web, App, DB)



→ Nginx talks to App
→ App talks to DB

→ DB is not exposed to the internet.

10. DEBUGGING NETWORK ISSUES

- ✓ Check container IP: `docker inspect <container>`
- ✓ Test connectivity: `docker exec -it <container> ping <host>`
- ✓ Check open ports: `docker port <container>`
- ✓ Inspect network: `docker network inspect <network>`
- ✓ Look at logs: `docker logs <container>`

PRO TIP

Good network design in Docker means secure, scalable and maintainable containerized applications.



Strong networks connect containers. Smart networks build production systems.



DOCKER COMPOSE FUNDAMENTALS

★ Define, Run and Orchestrate Multi-Container Applications ★



Docker Compose lets you define and run multi-container applications using a simple YAML file. One command to bring everything up!

1. WHAT IS DOCKER COMPOSE?

- ✓ Tool for defining and running multi-container applications.
- ✓ Uses a YAML file (compose.yaml) to configure services.
- ✓ Manages services, networks, volumes and dependencies.
- ✓ One command to start, stop and manage everything.



2. COMPOSE FILE (compose.yaml)

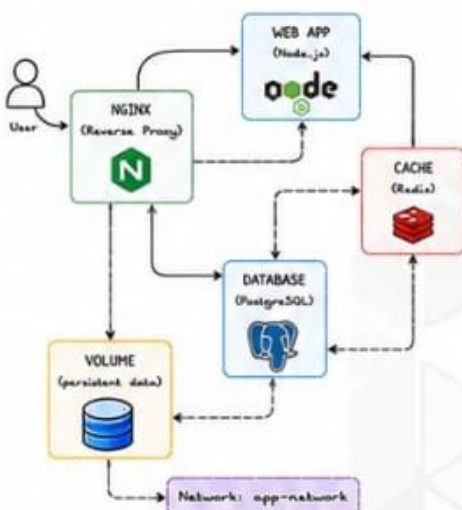
Structure of a Compose File:

```
version: '3.9'           # Compose File version
services:                 # All containers (services)
  web:                     # Service name
    image: nginx:latest   # Image to use
    ports:                 # Port mapping
      - "80:80"
    environment:           # Environment variables
      - KEY=value
    volumes:                # Volumes
      - ./data:/app/data
    depends_on:             # Start order
      - db
  networks:                # Networks
    app-network
  volumes:                 # Named volumes
    db-data:
```

3. KEY CONCEPTS

- Services** - Individual containers defined in compose file.
- Networks** - Containers communicate over networks.
- Volumes** - Data persistence between container restarts.
- Environment** - Pass configuration to containers.
- Depends_on** - Control startup order between services.

4. MULTI-CONTAINER APPLICATION ARCHITECTURE



5. EXAMPLE compose.yaml (PRODUCTION STYLE)

```
version: '3.9'
services:
  web:
    image: myapp:1.0
    container_name: web
    ports:
      - "80:80"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
      - DB_USER=postgres
      - DB_PASSWORD=secret
    depends_on:
      - db
      - redis
    networks:
      - app-network
    restart: unless-stopped
    volumes:
      - ./logs:/app/logs

  db:
    image: postgres:15
    container_name: db
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=secret
      - POSTGRES_DB=myapp
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-network
    restart: unless-stopped

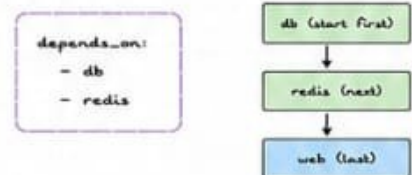
  redis:
    image: redis:7-alpine
    container_name: redis
    networks:
      - app-network
    restart: unless-stopped

networks:
  app-network:
    driver: bridge

volumes:
  db-data:
```

6. SERVICE DEPENDENCY (depends_on)

Ensures services start in the right order.



7. ENVIRONMENT VARIABLES

Pass configuration without hardcoding.

```
environment:
  - NODE_ENV=production
  - DB_HOST=db
  - DB_USER=postgres
  - DB_PASSWORD=secret
```

Use .env file for sensitive values.

8. VOLUMES IN COMPOSE

Persist data across container restarts.

```
volumes:
  - db-data:/var/lib/postgresql/data
  - ./logs:/app/logs
```

Named Volume (db-data)

Bind Mount (./logs)

9. COMPOSE WORKFLOW



11. COMMON COMPOSE COMMANDS

COMMAND	DESCRIPTION
<code>docker compose up</code>	Start all services in foreground
<code>docker compose up -d</code>	Start all services in detached mode (bg)
<code>docker compose down</code>	Stop and remove all services, networks
<code>docker compose ps</code>	List running services
<code>docker compose logs</code>	View logs from services
<code>docker compose exec <service> sh</code>	Run command in a service container
<code>docker compose build</code>	Build or rebuild services
<code>docker compose pull</code>	Pull latest images for services
<code>docker compose config</code>	Validate and view merged config

10. BEST PRACTICES

- ✓ Keep compose.yaml clean and readable.
- ✓ Use named volumes for production data.
- ✓ Use .env file for secrets and config.
- ✓ Use restart: unless-stopped for resilience.
- ✓ Pin image versions (avoid latest in prod).
- ✓ Use healthcheck in Dockerfile for critical services.
- ✓ Use depends_on for startup order, not readiness.

12. COMMON MISTAKES

- ✗ Forgetting to expose ports.
- ✗ Not using volumes (data loss!).
- ✗ Hardcoding secrets in compose.yaml.
- ✗ Using depends_on for readiness.
- ✗ Not cleaning unused containers/networks.

PRO TIP

Compose is perfect for local development, testing and small production workloads. For large scale, combine with orchestration tools like Kubernetes.



Test your compose stack before production deployment!



Docker Compose = define once, run anywhere.
Simplify multi-container apps with one command!





DOCKER REGISTRY

★ Docker Hub, ECR, ACR, GAR and Private Registries ★



A Docker Registry stores and distributes container images.
Push once, pull anywhere.

1. WHAT IS A DOCKER REGISTRY?

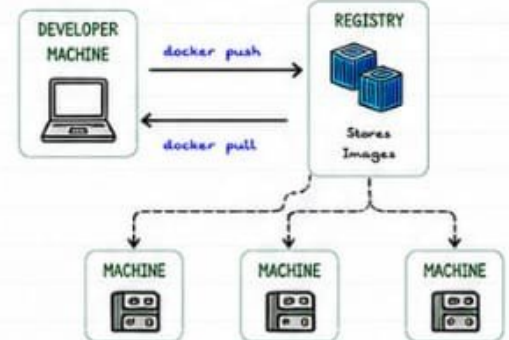
- ✓ A registry stores container images.
- ✓ Images are versioned using tags.
- ✓ You push images to a registry.
- ✓ You pull images from a registry to run on any machine.
- ✓ Public registries: Docker Hub, GHCR.
- ✓ Cloud registries: AWS ECR, Azure ACR, GCP GAR.
- ✓ You can also run your own private registry.



2. REGISTRY TYPES

TYPE	EXAMPLES
Public Registry	Docker Hub, GitHub Container Registry
Cloud Registry	AWS ECR, Azure ACR, Google GAR
Private Registry	Self-hosted with Docker Registry
On-Prem Registry	Harbor, Nexus, Artifactory

3. HOW REGISTRIES WORK



4. DOCKER HUB (PUBLIC)

- ✓ Default registry for Docker.
- ✓ Free for public repositories.
- ✓ Great for open source projects.
- ✓ Rate limits for anonymous users.

DEFAULT IMAGE NAME FORMAT

docker.io/username/image:tag

EXAMPLE

```

$ Pull from Docker Hub
$ docker pull nginx:latest
$ Push to Docker Hub
$ docker push myuser/myapp:1.0
  
```

5. AWS ECR (ELASTIC CONTAINER REGISTRY)

- ✓ Fully managed private registry by AWS.
- ✓ Secure, scalable and highly available.
- ✓ Integrates with IAM for access control.
- ✓ Used in CI/CD and production.

EXAMPLE

```

$ aws ecr get-login-password --region us-east-1 \
| docker login --username AWS --password-stdin \
123456789012.dkr.ecr.us-east-1.amazonaws.com

$ docker tag app:1.0 123456789012.dkr.ecr.us-east-1. \
amazonaws.com/myapp:1.0
$ docker push 123456789012.dkr.ecr.us-east-1. \
amazonaws.com/myapp:1.0
  
```

6. AZURE ACR (AZURE CONTAINER REGISTRY)

- ✓ Private registry service by Microsoft.
- ✓ Role-based access control (RBAC).
- ✓ Integration with Azure DevOps and AKS.
- ✓ Supports geo-replication.

EXAMPLE

```

$ az acr login --name myregistry
$ docker tag app:1.0 myregistry.azurecr.io/myapp:1.0
$ docker push myregistry.azurecr.io/myapp:1.0
  
```

7. GOOGLE GAR (GOOGLE ARTIFACT REGISTRY)

- ✓ Managed Docker registry by Google Cloud.
- ✓ Supports Docker, Maven, npm, Python, etc.
- ✓ Built-in vulnerability scanning.
- ✓ Works with GKE, Cloud Run, Cloud Build.

EXAMPLE

```

$ gcloud auth configure-docker us-docker.pkg.dev
$ docker tag app:1.0 us \
docker.pkg.dev/myproject/my-repo/myapp:1.0
$ docker push us-docker.pkg.dev/my \
project/my-repo/myapp:1.0
  
```

8. PRIVATE REGISTRY (SELF-HOSTED)

- ✓ Run your own registry inside your network.
- ✓ Full control over images and access.
- ✓ Ideal for enterprises with compliance needs.

RUN PRIVATE REGISTRY

```

$ docker run -d \
--name registry \
-p 5000:5000 \
-v /opt/registry/data:/var/lib/registry \
registry:2
  
```



Access at: <http://your-server-ip:5000>
Example: `docker pull your-ip:5000/myapp:1.0`

9. IMAGE TAGGING STRATEGIES

TAG	USE CASE
latest	Default stable version
v1.0.0	Production release
v1.0.0-rc1	Release candidate
dev	Development builds
sha-abcdeF	Immutable by commit SHA



Use specific tags in production, never "latest".

10. SEMANTIC VERSIONING

Follow SemVer for image tags.

FORMAT

MAJOR.MINOR.PATCH

EXAMPLES

```

1.0.0 → Initial release
1.1.0 → New Features
1.1.1 → Bug Fixes
2.0.0 → Breaking changes
  
```

11. AUTHENTICATION & SECURITY

- ✓ Always authenticate before pushing.
- ✓ Use IAM/RBAC for fine-grained access.
- ✓ Enable image scanning and vulnerability checks.
- ✓ Use private registries for sensitive images.
- ✓ Use short-lived tokens in CI/CD.

EXAMPLE: DOCKER LOGIN

```

$ docker login
Username: myuser
Password: *****
Login Succeeded
  
```



12. BEST PRACTICES

- ✓ Tag images properly (no latest in prod).
- ✓ Scan images before pushing.
- ✓ Use minimal base images.
- ✓ Keep images small and secure.
- ✓ Enable lifecycle policies (cleanup old images).
- ✓ Use private registries for sensitive data.

PRO TIP

A good registry strategy =
Better security, faster deployments,
and reliable production systems.



Build once. Push once. Run everywhere.
Registries make your containers portable and powerful.





ENVIRONMENT VARIABLES & SECRETS

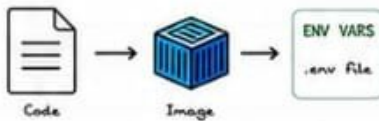
☆ Configure Applications the Right Way ☆



Keep your config flexible. Keep secrets safe.
Don't bake config or secrets into images.

1. WHY USE ENVIRONMENT VARIABLES?

- ✓ Keep config separate from code.
- ✓ Different values per environment.
- ✓ Easy to update without rebuilding image.
- ✓ Good for non-sensitive configuration.
- ✓ Used in Docker, Compose and Kubernetes.



2. WAYS TO SET ENV VARIABLES

METHOD	DESCRIPTION
Dockerfile ENV	Set default values in image.
docker run -e	Set at runtime.
docker-compose.yml environment	Set in Compose File.
.env file	Store values and reference in Compose.
env_file	Load variables from file.

3. DOCKERFILE ENV EXAMPLE

```

# Dockerfile
FROM node:18-alpine

ENV NODE_ENV=production
ENV PORT=3000
ENV APP_NAME=veriqta-app

WORKDIR /app
COPY . .
CMD ["node", "server.js"]
  
```

★ ENV in Dockerfile is baked into the image. Do not use for secrets!

4. RUNTIME ENV EXAMPLE

EXAMPLE

```

$ docker run -d \
-e NODE_ENV=production \
-e PORT=8080 \
-e DB_HOST=db \
-e DB_USER=veriqta \
-e DB_PASSWORD=secret \
myapp:1.0
  
```

★ Great for dynamic values in different environments.

5. COMPOSE ENVIRONMENT EXAMPLE

EXAMPLE (compose.yml)

```

services:
  web:
    image: myapp:1.0
    environment:
      - NODE_ENV=production
      - PORT=8080
      - DB_HOST=db
      - DB_USER=veriqta
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: veriqta
      POSTGRES_PASSWORD: secret
  
```

6. USING .ENV FILE

.env FILE EXAMPLE

```

NODE_ENV=production
PORT=8080
DB_HOST=db
DB_USER=veriqta
DB_PASSWORD=secret
API_URL=https://api.veriqta.com
  
```

★ In compose.yml:
env_file:
- .env

7. ENV_FILE (MULTIPLE FILES)

EXAMPLE (compose.yml)

```

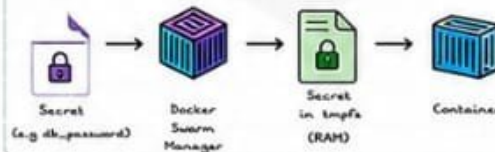
services:
  web:
    image: myapp:1.0
    env_file:
      - .env
      - .env.production
    environment:
      - NODE_ENV=production
  
```

★ Later files override earlier ones.

8. DOCKER SECRETS (SENSITIVE DATA)

- ✓ Use Docker Secrets for sensitive data.
- ✓ Secrets are not exposed in ENV or logs.
- ✓ Stored in memory (tmpfs) on the container.
- ✓ Supported in Swarm mode.

HOW IT WORKS



9. CREATE & USE SECRET (SWARM)

```

# Create secret
$ echo "mysecretpass" | docker secret create db_pass -
# Use in service
$ docker service create \
  --name myapp \
  --secret db_pass \
  myapp:1.0
  
```

SECRET FILE INSIDE CONTAINER

SECRET NAME	MOUNT PATH
db_pass	/run/secrets/db_pass

★ App reads secret from file, not ENV.

10. BEST PRACTICES

- ✓ Use ENV for config, not secrets.
- ✓ Use .env file for local development.
- ✓ Use Docker Secrets for production secrets.
- ✓ Never commit .env or secrets to Git.
- ✓ Use least privilege and rotate secrets.
- ✓ Keep secrets out of logs and images.
- ✓ Use multiple env files for each environment.

PRO TIP

Always treat secrets like passwords to your infrastructure.



11. COMMON MISTAKES

- ✗ Hardcoding secrets in Dockerfile.
- ✗ Committing .env files to Git.
- ✗ Using ENV for passwords or API keys.
- ✗ Printing secrets in logs.
- ✗ Not using secrets in production.
- ✗ Forgetting to rotate secrets.



One exposed secret can compromise everything!

12. QUICK REFERENCE

TASK	COMMAND / METHOD
Set env in Dockerfile	ENV KEY=value
Set env at runtime	docker run -e KEY=value
Set env in Compose	environment: - KEY=value
Load from .env	env_file: - .env
Use Docker Secret	docker secret create & mount
View env inside container	docker exec -it <id> env
View secrets inside container	ls -l /run/secrets



Configure smart. Keep secrets safe. Deploy with confidence.





DOCKER LOGGING

☆ Collect, Store and Analyze Container Logs ☆



Logs are the first place to look when something breaks.
Good logging = fast debugging + happy users.

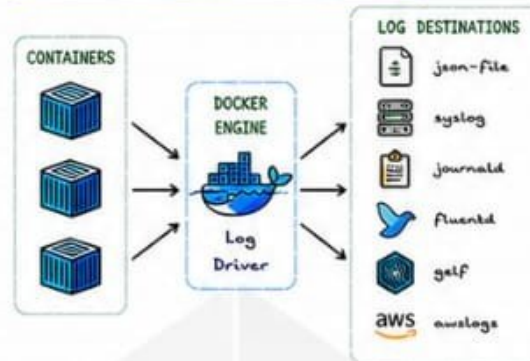
1. WHY LOGGING MATTERS

- ✓ Understand application behavior.
- ✓ Debug issues quickly.
- ✓ Monitor performance and errors.
- ✓ Audit and compliance.
- ✓ Centralize logs for the whole stack.



No logs = No visibility.
No visibility = No control.

2. DOCKER LOGGING ARCHITECTURE



3. DEFAULT LOGGING DRIVER

json-file

- ✓ Default in Docker.
- ✓ Stores logs in JSON format.
- ✓ Easy to read but can consume disk space.
- ✓ Good for small environments.

EXAMPLE LOG FILE

```
{
  "log": "Server started",
  "stream": "stdout",
  "time": "2025-06-03T10:15:10.123456789Z"
}
```

4. VIEWING LOGS

COMMAND	DESCRIPTION
<code>docker logs <container></code>	View logs from container
<code>docker logs -f <container></code>	Follow logs in real time
<code>docker logs --since 1h <c></code>	Show logs from last 1 hour
<code>docker logs --tail 100 <c></code>	Show last 100 lines
<code>docker logs -f --tail 200 <c></code>	Follow last 200 lines

5. LOGGING DRIVERS

DRIVER	USE CASE
json-file	Default, local dev, small apps
syslog	Send logs to syslog server
journald	Systemd based systems
fluentd	Log aggregation (Fluentd)
gelf	Graylog logging
awslogs	AWS CloudWatch Logs
splunk	Send logs to Splunk
none	Disable logging (not recommended)

6. SET LOGGING DRIVER

At container level:

```
docker run -d \
  --name myapp \
  --log-driver syslog \
  --log-opt syslog-address=udp://192.168.1.100:514 \
  myapp:1.0
```

At daemon level (/etc/docker/daemon.json):

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

7. JSON-FILE OPTIONS

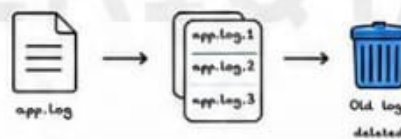
OPTION	DESCRIPTION
max-size	Max size of a log file (e.g. 10m)
max-file	Max number of log files to keep
compress	Compress rotated logs (Docker 24+)
labels	Add container labels to log entry
env	Add environment variables to log



Configure these in daemon.json under log-opts.

8. LOG ROTATION

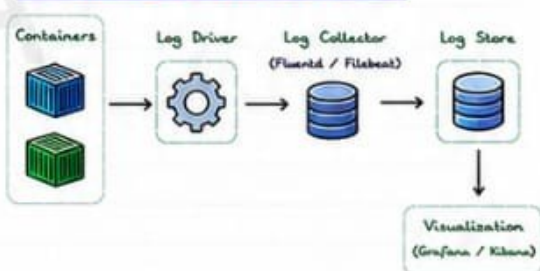
Prevent log files from growing forever.



EXAMPLE

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

9. CENTRALIZED LOGGING STACK



10. BEST PRACTICES

- ✓ Use appropriate logging driver for your environment.
- ✓ Rotate logs to avoid disk full.
- ✓ Centralize logs in production.
- ✓ Never log sensitive data (passwords, tokens, keys).
- ✓ Use structured logging (JSON) when possible.
- ✓ Monitor logs and set up alerts.

PRO TIP

Logs are your application's diary.
Read them like a detective!



11. COMMON ISSUES

- ✗ Container logs not showing up.
- ✗ Logs consuming all disk space.
- ✗ Wrong logging driver configured.
- ✗ Centralized logs delayed.
- ✗ Sensitive data leaked in logs.



Bad logging = slow debugging.
Good logging = fast resolution.

12. QUICK REFERENCE

TASK	COMMAND
View logs	<code>docker logs <container></code>
Follow logs	<code>docker logs -f <container></code>
Last 100 lines	<code>docker logs --tail 100 <container></code>
Since 1 hour	<code>docker logs --since 1h <container></code>
Set log driver	<code>--log-driver <driver></code>
Set log options	<code>--log-opt <key>=<value></code>
Configure daemon	<code>/etc/docker/daemon.json</code>



Collect smart. Store safe. Analyze fast. Log everything that matters.





DOCKER SECURITY FUNDAMENTALS

☆ Secure Images. Secure Containers. Secure Everything. ☆



Security is not a feature, it's a practice.

Build secure images, run secure containers, protect your systems.

1. WHY SECURITY MATTERS

- ✓ Prevent vulnerabilities and exploits.
- ✓ Protect data and secrets.
- ✓ Reduce attack surface.
- ✓ Ensure compliance and trust.
- ✓ Secure by design, not by accident.



A small misconfiguration today can be a major incident tomorrow.

2. DOCKER SECURITY PILLARS



3. IMAGE SECURITY BEST PRACTICES

- ✓ Use official or trusted base images.
- ✓ Keep base images updated.
- ✓ Scan images for vulnerabilities.
- ✓ Remove unnecessary packages.
- ✓ Use multi-stage builds to keep images small.
- ✓ Do not store secrets in images.

EXAMPLE: SCAN IMAGE

```
$ docker scan myapp:1.0
✓ No critical vulnerabilities
✓ 2 High
✓ 5 Medium
✓ 10 Low
```



Scan early. Scan often.
Scan everything.

4. CONTAINER SECURITY BEST PRACTICES

- ✓ Run containers as non-root user.
- ✓ Drop unnecessary Linux capabilities.
- ✓ Use read-only root filesystem.
- ✓ Limit resource usage (CPU, Memory, PIDs).
- ✓ Do not allow privileged containers.
- ✓ Use security profiles (AppArmor/SELinux).
- ✓ Keep containers stateless.



5. DOCKER DAEMON SECURITY

- ✓ Keep Docker Engine updated.
- ✓ Restrict access to Docker socket.
- ✓ Do not expose Docker socket.
- ✓ Use TLS for daemon communication.
- ✓ Use rootless mode when possible.
- ✓ Enable user namespace remap.

EXAMPLE: DOCKER DAEMON TLS

```
$ dockerd \
  --tlsv1verify \
  --tlscacert=ca.pem \
  --tlscert=server-cert.pem \
  --tlskey=server-key.pem \
  -Htcp://0.0.0.0:2376
```

6. SECRETS MANAGEMENT

- ✓ Never store secrets in Dockerfile or code.
- ✓ Use Docker Secrets for sensitive data.
- ✓ Mount secrets at runtime.
- ✓ Limit secret exposure.

DOCKER SECRETS FLOW

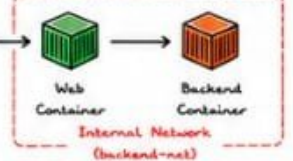


7. SCAN & COMPLIANCE TOOLS

TOOL	USE CASE
Trivy	Vulnerability scanning
Grype	Fast vulnerability scanner
Docker Bench	Docker security benchmark
Snyk	Vuln scanning + remediation
Clair	Static analysis of images

8. NETWORK SECURITY

- ✓ Use user-defined bridge networks.
- ✓ Do not expose unnecessary ports.
- ✓ Use internal networks for backend services.
- ✓ Restrict container communication.



9. COMMON SECURITY RISKS

- ✗ Running as root.
- ✗ Using outdated images.
- ✗ Exposing Docker socket.
- ✗ Storing secrets in images.
- ✗ Privileged containers.
- ✗ Weak network isolation.
- ✗ Unscanned images.
- ✗ Overly permissive volumes.

10. BEST PRACTICES SUMMARY

- ✓ Build minimal and secure images.
- ✓ Scan images regularly.
- ✓ Run containers as non-root.
- ✓ Use least privilege.
- ✓ Protect secrets.
- ✓ Keep Docker and host updated.
- ✓ Monitor and audit your containers.

PRO TIP

Security is a continuous process, not a one-time setup.



11. QUICK COMMANDS REFERENCE

COMMAND	DESCRIPTION
<code>docker scan <image></code>	Scan image for vulnerabilities
<code>docker run --rm -u 1000:1000</code>	Run as non-root user
<code>docker run --read-only</code>	Run with read-only root FS
<code>docker secrets ls</code>	List secrets in swarm
<code>docker system info</code>	Check Docker security info
<code>docker info grep -i security</code>	View security settings



Secure from the base image to the running container.
A secure container today, a safe production tomorrow.





PERFORMANCE OPTIMIZATION

★ Build Faster. Run Lighter. Scale Better. ★



Performance is not an accident.

Optimize images, containers and resources for speed and efficiency.

1. WHY PERFORMANCE MATTERS

- ✓ Faster builds and deployments.
- ✓ Smaller images = less bandwidth & storage.
- ✓ Better resource utilization.
- ✓ Lower cost in cloud environments.
- ✓ Improved user experience.



A small improvement today
saves big resources tomorrow.

2. IMAGE SIZE OPTIMIZATION

- ✓ Use smaller base images (alpine, slim).
- ✓ Use multi-stage builds.
- ✓ Remove unnecessary packages.
- ✓ Clean package manager cache.
- ✓ Use .dockerignore.
- ✓ Use distroless images where possible.



Fat Image
1.2 GB

Optimize



Optimized Image
120 MB



Faster Pull
& Start

3. MULTI-STAGE BUILD (BEST PRACTICE)



EXAMPLE: DOCKERFILE

```
# Stage 1: Build
FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Stage 2: Runtime
FROM alpine:3.19
WORKDIR /app
COPY --from=builder /app/myapp .
CMD ["/myapp"]
```

4. CONTAINER RUNTIME OPTIMIZATION

- ✓ Run only what you need.
- ✓ Use appropriate resource limits.
- ✓ Avoid running as root.
- ✓ Use read-only file system when possible.
- ✓ Disable unnecessary services.

RUN CONTAINER OPTIMIZED

```
$ docker run -d \
--name myapp \
--cpus=0.5 \
--memory=512m \
--read-only \
--security-opt no-new-privileges:true \
--user 1000:1000 \
myapp:1.0
```

5. RESOURCE LIMITS (CPU & MEMORY)

FLAG	DESCRIPTION
--cpus	Limit CPU (e.g. 0.5 = 50%)
--cpu-shares	Relative CPU weight
--memory	Memory limit (e.g. 512m)
--memory-swap	Total memory (mem + swap)
--pids-limit	Limit number of processes

RESOURCES IN BALANCE



Not too much.



Not too little.



Just right!

6. LAYER CACHING STRATEGIES

- ✓ Order Dockerfile instructions smartly.
- ✓ Copy package files first.
- ✓ Leverage build cache.
- ✓ Avoid changing files unnecessarily.

GOOD ORDER

```
COPY package*.json /
RUN npm install
COPY . .
RUN npm run build
```

BAD ORDER

```
COPY . .
RUN npm install
RUN npm run build
```



Change code often, not dependencies!
Better cache = faster builds.

7. STORAGE & IO OPTIMIZATION

- ✓ Use volumes for persistent data.
- ✓ Avoid writing to container layer.
- ✓ Use tmpfs for temporary data.
- ✓ Choose faster storage (SSD).

VOLUME > CONTAINER LAYER



Container
(Ephemeral)



Volume
(Persistent)



Data
(Safe)

8. NETWORK PERFORMANCE

- ✓ Use bridge network for local apps.
- ✓ Use overlay for multi-host apps.
- ✓ Minimize cross-container traffic.
- ✓ Enable keep-alive & connection pooling.
- ✓ Use host network only when necessary.

NETWORK CHOICES



bridge



overlay



host

9. MONITORING & METRICS

- ✓ Monitor CPU, Memory, Disk, Network.
- ✓ Use cAdvisor, Prometheus, Grafana.
- ✓ Alert before issues happen.

OBSERVABILITY STACK



Docker /
cAdvisor



Prometheus



Grafana



Insights
& Alerts

10. COMMON BOTTLENECKS

- ✗ Huge images & unnecessary layers.
- ✗ Too many containers per host.
- ✗ Memory leaks in applications.
- ✗ High disk I/O.
- ✗ Network latency & DNS issues.
- ✗ No resource limits.
- ✗ Logging overhead.

PRO TIP

Measure first.
Optimize what matters.
Never guess!



11. OPTIMIZATION CHECKLIST

- ✓ Small image size.
- ✓ Multi-stage build used.
- ✓ Resources limited.
- ✓ Non-root user.
- ✓ Read-only filesystem.
- ✓ Healthcheck configured.
- ✓ Logging optimized.
- ✓ Monitoring enabled.
- ✓ Secrets managed properly.
- ✓ Performance tested.

CHECKLIST



12. QUICK COMMANDS REFERENCE

COMMAND	PURPOSE
<code>docker image ls</code>	List images & sizes
<code>docker system df</code>	Show docker disk usage
<code>docker stats</code>	Live container resource usage
<code>docker inspect <id></code>	Detailed container info
<code>docker top <id></code>	View running processes
<code>docker exec -it <id> sh</code>	Inspect container
<code>docker system prune -a</code>	Clean unused resources



Optimize for performance. Optimize for scale.
Small improvements today = Massive wins tomorrow.





DOCKER TROUBLESHOOTING FUNDAMENTALS

☆ Find Problems Fast. Diagnose Accurately. Fix with Confidence. ☆



Every engineer faces issues. Great engineers troubleshoot systematically.
Identify. Investigate. Isolate. Fix. Verify. Learn.

1. TROUBLESHOOTING MINDSET

- ✓ Stay calm. Don't guess.
- ✓ Reproduce the issue.
- ✓ Gather information.
- ✓ Isolate the root cause.
- ✓ Fix the problem.
- ✓ Verify the fix.
- ✓ Document what you learned.



Systematic > Random
Evidence > Assumption

2. TROUBLESHOOTING WORKFLOW



1. Identify: What is broken?
2. Gather Info: Collect logs & data
3. Isolate: Narrow down the cause
4. Fix: Apply the solution
5. Verify: Confirm it's resolved
6. Learn: Document & improve

3. ESSENTIAL INSPECTION COMMANDS

COMMAND	PURPOSE
<code>docker ps -a</code>	List all containers
<code>docker inspect <id></code>	Inspect container details
<code>docker logs <id></code>	View container logs
<code>docker exec -it <id> sh</code>	Access running container
<code>docker top <id></code>	View running processes
<code>docker stats</code>	Real-time resource usage
<code>docker info</code>	Docker system info
<code>docker events</code>	Real-time Docker events
<code>docker system df</code>	Disk usage summary



Inspect everything.
Logs never lie.

4. COMMON CONTAINER ISSUES

ISSUE	POSSIBLE CAUSES
Container won't start	Bad config, missing deps, port conflict, crash
Container exits immediately	App error, wrong CMD, missing files
High CPU usage	Infinite loop, heavy process, resource leak
High Memory usage	Memory leak, large cache, poor limits
Cannot connect	Wrong port, network issue, firewall
Files/Data missing	Volume not mounted, permission issues

5. CHECK CONTAINER LOGS

```
$ docker logs myapp
$ docker logs -f myapp # follow logs
$ docker logs --tail 100 myapp
$ docker logs --since 10m myapp
```



Logs are your first clue.
Always start here.

6. ACCESS & DEBUG CONTAINER

```
$ docker exec -it myapp sh
$ docker exec -it myapp bash
$ docker exec myapp env
$ docker exec myapp cat /var/log/app.log
```

USE CASES

- ✓ Check env variables
- ✓ Inspect file system
- ✓ Run diagnostic commands
- ✓ Test connectivity (ping, curl, nc)



7. NETWORK TROUBLESHOOTING

CHECK NETWORKS

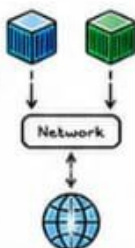
```
$ docker network ls
$ docker network inspect <network>
```

COMMON ISSUES

- ✗ Container not on the same network
- ✗ Wrong port published
- ✗ Service name resolution failure
- ✗ Firewall blocking traffic

TEST CONNECTIVITY

```
$ docker exec -it <container> ping <host>
$ docker exec -it <container> curl http://<service>:<port>
```



8. STORAGE & VOLUME ISSUES

CHECK VOLUMES

```
$ docker volume ls
$ docker volume inspect <volume>
$ docker system df -v
```

COMMON PROBLEMS

- ✗ Volume not mounted
- ✗ Wrong path
- ✗ Permission denied
- ✗ Disk full

CHECK DISK USAGE

```
$ df -h
$ docker system df
$ docker system prune -a # cleanup
```



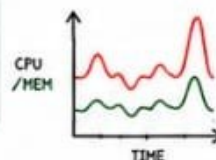
9. PERFORMANCE DEBUGGING

CHECK RESOURCES

```
$ docker stats
$ docker top <container>
$ docker inspect <container> --format '{{HostConfig.Memory}}'
```

LOOK FOR

- ✓ CPU spikes
- ✓ Memory leaks
- ✓ Disk I/O bottlenecks
- ✓ Too many restarts



10. RESTART & HEALTH CHECKS

CHECK RESTART POLICY

```
$ docker inspect <container> --format '{{HostConfig.RestartPolicy.Name}}'
```

CHECK HEALTH STATUS

```
$ docker inspect <container> --format '{{State.Health.Status}}'
```

COMMON RESTART REASONS

- ✗ Application crash
- ✗ Out of memory (OOMKilled)
- ✗ Health check failing
- ✗ Dependency not ready



11. RESET & RECOVERY

RESTART CONTAINER

```
$ docker restart <container>
```

REMOVE & RECREATE

```
$ docker rm -f <container>
$ docker run ...
```

PRUNE UNUSED

```
$ docker system prune -a --volumes
```



Prune regularly.
Keep your system clean.



12. QUICK TROUBLESHOOTING CHECKLIST

- ✓ Check container status (`docker ps -a`)
- ✓ Check logs (`docker logs`)
- ✓ Inspect container (`docker inspect`)
- ✓ Access container (`docker exec`)
- ✓ Check resources (`docker stats`)
- ✓ Check network (`docker network ls`)
- ✓ Check volumes (`docker volume ls`)
- ✓ Fix the issue
- ✓ Verify the fix
- ✓ Document the root cause & solution



Debug with data. Fix with logic. Learn with every incident.
Every problem solved makes you a **better engineer**.





REAL PRODUCTION FAILURE CASE STUDIES

☆ Learn from Real Incidents. Avoid Real Disasters. ☆



Every outage has a root cause.

Study the failures. Implement the fixes. Prevent the next incident.

1. OUT OF DISK SPACE MELTDOWN

DISK FULL
100%

Incident: Production API slow → Errors → Complete outage

Timeline:

- 09:10 Latency started increasing
- 09:20 Errors spiked
- 09:25 Containers stopped
- 09:30 Service down

Root Cause:

Docker logs and unused images filled the disk.

Impact:

- API unavailable for 15 minutes
- Users received 500 errors
- SLA breached

Fix / Prevention:

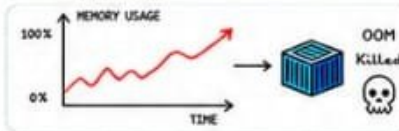
- ☒ Set up log rotation
- ☒ Automated cleanup (prune)
- ☒ Disk monitoring & alerts



Lessons Learned:

Monitor disk. Clean up regularly. Automation prevents outages.

2. MEMORY LEAK IN CONTAINER



Incident: Container killed repeatedly (OOMKilled)

Timeline:

- 14:05 Memory usage climbing
- 14:20 Container restarted
- 14:25 Restart loop begins
- 14:30 Service unstable

Root Cause:

Application memory leak + no memory limits.

Impact:

- Frequent restarts
- High CPU & memory usage
- Unstable service

Fix / Prevention:

- ☒ Set memory limits (-m)
- ☒ Fix memory leak in code
- ☒ Add monitoring & alerts



Lessons Learned:

Set limits on everything. Monitor. Don't assume.

3. WRONG IMAGE VERSION DEPLOYED



Pushed → Wrong Version → Rollback v1.2.2

Incident: New deployment broke the application

Timeline:

- 11:00 Deployed new image
- 11:02 Errors reported
- 11:05 Issue identified
- 11:10 Rolled back

Root Cause:

Wrong image tag used (latest instead of stable).

Impact:

- Application errors
- User requests failed
- Emergency rollback

Fix / Prevention:

- ☒ Use explicit image tags
- ☒ Implement CI/CD checks
- ☒ Rollback strategy in place



Lessons Learned:

Never use latest in production. Always pin your versions.

4. NETWORK MISCONFIGURATION



Cannot connect



Incident: Web app couldn't connect to database

Timeline:

- 16:40 Deployment completed
- 16:42 DB connection errors
- 16:50 Issue investigated
- 17:05 Network fixed

Root Cause:

Containers on different networks.

Impact:

- Application down
- Database unreachable
- Revenue impact

Fix / Prevention:

- ☒ Use user-defined bridge network
- ☒ Validate network config
- ☒ Add health checks



Lessons Learned:

Containers must be on the same network to communicate.

5. UNHEALTHY CONTAINER TAKING TRAFFIC



Incident: Traffic sent to unhealthy containers

Timeline:

- 10:15 High error rate
- 10:18 Unhealthy container still receiving traffic
- 10:20 Users affected
- 10:25 Health check fixed

Root Cause:

Missing or incorrect health check configuration.

Impact:

- High error rate
- Slow responses
- User experience degraded

Fix / Prevention:

- ☒ Implement proper health checks
- ☒ Use --health-cmd & --health-interval
- ☒ Configure load balancer correctly



Lessons Learned:

Health checks save users. Fail fast. Remove bad instances.

6. VOLUME MOUNT OVERWRITE



Image Data → Volume Mount → Data Lost

Incident: Application data disappeared after deploy

Timeline:

- 08:00 New container deployed
- 08:05 Data missing
- 08:10 Issue investigated
- 08:20 Rollback & recovery

Root Cause:

Volume mount overwrote container data.

Impact:

- Data loss
- Application misconfigured
- Recovery effort required

Fix / Prevention:

- ☒ Understand volume mount behavior
- ☒ Backup data
- ☒ Use named volumes correctly



Lessons Learned:

Volumes override image data. Know how Docker mounts.

COMMON THEMES FROM FAILURES

- ☒ Lack of monitoring & alerts
- ☒ No resource limits
- ☒ Poor logging & visibility
- ☒ Misconfiguration
- ☒ No automation
- ☒ Human error
- ☒ No rollback strategy
- ☒ Ignoring best practices



GENERAL PREVENTION STRATEGIES

1. Monitor everything (CPU, Memory, Disk, Network).
2. Set resource limits on all containers.
3. Use health checks.
4. Keep images updated and secure.
5. Automate cleanup & maintenance.
6. Use versioned images, never latest.
7. Test before production.
8. Have rollback plan.
9. Backup important data.
10. Document everything.



KEY TAKEAWAY



Failures are inevitable. Outages are optional.

Prepare.
Monitor.
Automate.
Recover.
Repeat.

That's how you stay in production.

Your goal is not zero failures. Your goal is fast recovery.
Learn. Improve. Prevent. Repeat.

50 DOCKER INTERVIEW QUESTIONS

★ Practice. Prepare. Perform. ★



These questions cover Docker fundamentals to production.
Know the concepts. Understand the why behind every answer.

1. DOCKER BASICS (1-10)

1. What is Docker?
2. How is Docker different from Virtual Machines?
3. What is a container?
4. What is a Docker image?
5. What is a Dockerfile?
6. How do you build a Docker image?
7. How do you run a container?
8. What is the difference between CMD and ENTRYPOINT?
9. How do you stop and remove containers?
10. What is Docker Hub?

★ Understand the core building blocks.
Strong basics = Strong Docker engineer.

2. IMAGES & CONTAINERS (11-20)

11. What is the difference between ADD and COPY?
12. What is the difference between RUN and CMD?
13. How do you tag a Docker image?
14. How do you list images on your system?
15. How do you inspect an image?
16. What is a container lifecycle?
17. What happens when a container stops?
18. How do you remove unused images?
19. What is a base image?
20. How do you use a custom Docker image?

★ Images are blueprints.
Containers are running instances.

3. DOCKER STORAGE & VOLUMES (21-30)

21. What is a Docker volume?
22. How is a volume different from a bind mount?
23. How do you create a volume?
24. How do you list volumes?
25. How do you remove a volume?
26. What is Docker storage driver?
27. What is the default storage driver?
28. How do you persist data in a container?
29. What happens to data when a container is removed?
30. How do you share data between containers?

★ Data is critical.
Always persist data the right way.

4. NETWORKING (31-35)

31. What is Docker networking?
32. What are the default network types?
33. How do you create a custom bridge network?
34. How do containers communicate?
35. What is the difference between bridge network and host network?

DOCKER NETWORK TYPES

Bridge (default)	
Host	
None	
Overlay (Swarm)	
Macvlan	

5. DOCKER COMPOSE (36-40)

36. What is Docker Compose?
37. What is the purpose of docker-compose.yml?
38. How do you start services with Docker Compose?
39. How do you scale services in Compose?
40. How do you link services in Compose?

SAMPLE COMPOSE COMMANDS

\$ docker compose up	Start all services
\$ docker compose up -d	Start in detached mode
\$ docker compose down	Stop and remove services
\$ docker compose ps	List running services

6. DOCKER REGISTRY (41-45)

41. What is a Docker registry?
42. What is Docker Hub?
43. How do you push an image to Docker Hub?
44. How do you pull an image from a registry?
45. How do you use a private registry?

PRIVATE REGISTRY OPTIONS



7. SECURITY (46-48)

46. How do you scan Docker images for vulnerabilities?
47. How do you run containers as non-root user?
48. What are Docker security best practices?

SECURITY BEST PRACTICES



- ✓ Use official base images
- ✓ Scan images regularly
- ✓ Run as non-root user
- ✓ Keep Docker updated
- ✓ Use secrets, not environment variables

8. ADVANCED & PRODUCTION (49-50)

49. How do you monitor Docker containers in production?
50. What are the key things to consider before deploying Docker in production?

PRODUCTION CHECKLIST

- ✓ Logging & monitoring
- ✓ Health checks
- ✓ Resource limits
- ✓ Security hardening
- ✓ Backup & recovery
- ✓ Image versioning
- ✓ Disaster recovery plan



INTERVIEW TIPS

- ✓ Explain with examples.
- ✓ Think out loud.
- ✓ Be honest if you don't know.
- ✓ Focus on fundamentals.
- ✓ Show real-world experience.



PRO TIP

It's not about memorizing answers.
It's about understanding concepts
and applying them in real-world
scenarios.

KEY AREAS TO MASTER



Docker
Fundamentals



Images &
Containers



Storage &
Volumes



Networking



Compose



Registry



Security



Production
Readiness



Be prepared. Stay calm. Think logically.
You've got this. Go build something amazing with Docker!





DOCKER CHEAT SHEET

☆ Common Commands. Quick Reference. Everyday Use. ☆



Keep this page close. These commands are your everyday tools.
Practice them. Master them. Use them.

1. IMAGE COMMANDS

COMMAND	DESCRIPTION
<code>docker pull <image></code>	Pull an image from registry
<code>docker images</code>	List all local images
<code>docker rmi <image-id></code>	Remove an image
<code>docker rmi \$(docker images -q)</code>	Remove all images
<code>docker tag <image> <name:tag></code>	Tag an image
<code>docker push <name:tag></code>	Push image to registry
<code>docker history <image></code>	Show image history
<code>docker inspect <image></code>	Inspect image details

2. CONTAINER COMMANDS

COMMAND	DESCRIPTION
<code>docker run <image></code>	Run a new container
<code>docker run -d <image></code>	Run in detached mode
<code>docker run -it <image> bash</code>	Run interactive shell
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers
<code>docker start <id></code>	Start a stopped container
<code>docker stop <id></code>	Stop a running container
<code>docker restart <id></code>	Restart a container
<code>docker rm <id></code>	Remove a stopped container
<code>docker rm -f <id></code>	Force remove container
<code>docker logs <id></code>	View container logs
<code>docker exec -it <id> bash</code>	Execute command in container

3. CONTAINER INSPECTION

COMMAND	DESCRIPTION
<code>docker inspect <id></code>	Inspect container details
<code>docker top <id></code>	View running processes
<code>docker stats <id></code>	View resource usage
<code>docker diff <id></code>	Show changes in container
<code>docker port <id></code>	List port mappings
<code>docker cp <id>:/path /</code>	Copy from container
<code>docker cp /file <id>:/path</code>	Copy to container



QUICK TIP

- ☒ Use `-d` for detached mode
- ☒ Use `-it` for interactive mode
- ☒ Use `--rm` to auto remove
- ☒ Use `docker logs -f <id>` to follow logs

4. VOLUME COMMANDS

COMMAND	DESCRIPTION
<code>docker volume ls</code>	List all volumes
<code>docker volume create <name></code>	Create a volume
<code>docker volume inspect <name></code>	Inspect volume
<code>docker volume rm <name></code>	Remove a volume
<code>docker run -v <name>:/data <image></code>	Mount volume in container
<code>docker run -v \$(pwd)/app <image></code>	Bind mount current directory

VOLUME TYPES

- Named Volumes (Managed by Docker)
- Bind Mounts (Host Path)
- tmpfs Mounts (Memory based)



5. NETWORK COMMANDS

COMMAND	DESCRIPTION
<code>docker network ls</code>	List all networks
<code>docker network create <name></code>	Create a network
<code>docker network inspect <name></code>	Inspect network
<code>docker network rm <name></code>	Remove a network
<code>docker run --network <name> <image></code>	Run container in network
<code>docker network connect <net> <id></code>	Connect container to network
<code>docker network disconnect <net> <id></code>	Disconnect container

DEFAULT NETWORK: BRIDGE



6. DOCKER COMPOSE COMMANDS

COMMAND	DESCRIPTION
<code>docker compose up</code>	Start services
<code>docker compose up -d</code>	Start in detached mode
<code>docker compose down</code>	Stop and remove services
<code>docker compose ps</code>	List running services
<code>docker compose logs</code>	View logs
<code>docker compose logs -f</code>	Follow logs
<code>docker compose build</code>	Build services
<code>docker compose pull</code>	Pull latest images
<code>docker compose restart</code>	Restart services

COMPOSE FILE



- ☒ `compose.yaml`
- ☒ Services
- ☒ Networks
- ☒ Volumes
- ☒ Environment

7. SYSTEM COMMANDS

COMMAND	DESCRIPTION
<code>docker system df</code>	Show disk usage
<code>docker system prune</code>	Remove unused data
<code>docker system prune -a</code>	Remove all unused images
<code>docker info</code>	Show system information
<code>docker version</code>	Show Docker version

8. REGISTRY COMMANDS

COMMAND	DESCRIPTION
<code>docker login</code>	Login to registry
<code>docker logout</code>	Logout from registry
<code>docker search <image></code>	Search images
<code>docker pull <name:tag></code>	Pull tagged image
<code>docker push <name:tag></code>	Push tagged image

9. FILE & BUILD COMMANDS

COMMAND	DESCRIPTION
<code>docker build -t <name> .</code>	Build image from Dockerfile
<code>docker build -f <file> -t <name> .</code>	Build using custom Dockerfile
<code>docker save <image> -o file.tar</code>	Save image to tar file
<code>docker load -i file.tar</code>	Load image from tar file
<code>docker cp <src> <dest></code>	Copy files/folders

SAFE CLEANUP



- ☒ `docker system prune`
- ☒ `docker volume prune`
- ☒ `docker image prune -a`
- ☒ `docker container prune`

POPULAR REGISTRIES



Docker Hub



AWS ECR



Azure ACR



Google GCR



BUILD CONTEXT TIPS

- ☒ Keep `.dockerignore` updated
- ☒ Use specific base images
- ☒ Keep layers small
- ☒ Cache dependencies



Learn the commands. Use them daily.
Speed comes with practice. Mastery comes with repetition.





DOCKER PRODUCTION READINESS CHECKLIST

☆ Your Final Checklist Before You Go Live ☆



Use this checklist before shipping any Dockerized application to production.
Tick everything. Review everything. Then deploy with confidence.

1. IMAGES & BUILDS

- ✓ Use official base images (Alpine/Distroless).
- ✓ Images are minimal and optimized.
- ✓ Multi-stage build implemented.
- ✓ No secrets in Dockerfile.
- ✓ .dockerignore file is used.
- ✓ Image tagged with version (not latest).
- ✓ Image scanned for vulnerabilities.
- ✓ Image pushed to secure registry.



PRO TIP

Small, secure images
build faster and run safer.



2. CONTAINERS & RUNTIME

- ✓ Run as non-root user.
- ✓ Read-only root filesystem used.
- ✓ Only necessary ports exposed.
- ✓ Healthcheck configured.
- ✓ Resource limits set (CPU & Memory).
- ✓ Restart policy defined.
- ✓ Environment variables managed.
- ✓ Secrets handled securely.
- ✓ Container logs configured.
- ✓ Containers run in isolated networks.
- ✓ No unnecessary capabilities added.



REMEMBER

A container should have everything
it needs and nothing it doesn't.

3. STORAGE & DATA

- ✓ Volumes used for persistent data.
- ✓ Named volumes (not anonymous).
- ✓ Bind mounts used intentionally.
- ✓ Data backed up regularly.
- ✓ Volume permissions set correctly.
- ✓ No critical data stored in container.

VOLUME TYPES

- Named Volumes (Managed)
- Bind Mounts (Host Path)
- tmpfs Mounts (Memory)



NOTE

Containers are ephemeral.
Data must survive.

4. NETWORKING

- ✓ Use user-defined bridge networks.
- ✓ Containers are not on default bridge.
- ✓ Network communication is restricted.
- ✓ Unnecessary ports are closed.
- ✓ DNS and service discovery configured.
- ✓ Overlay networks used for multi-host.

NETWORK MODES



bridge
(default)



none
(isolated)



host
(no isolation)



overlay
(multi-host)

5. SECURITY

- ✓ Images scanned for vulnerabilities.
- ✓ Run containers as non-root.
- ✓ No hardcoded secrets.
- ✓ Secrets stored in Docker Secrets.
- ✓ Use least privilege principle.
- ✓ Drop all unnecessary capabilities.
- ✓ Read-only filesystem enabled.
- ✓ Regular security updates applied.
- ✓ Audit and monitor container activity.



SECURITY PRINCIPLES

- Minimize attack surface.
- Limit privileges.
- Protect secrets.
- Monitor everything.

6. LOGGING & MONITORING

- ✓ Logging driver configured.
- ✓ Logs centralized (ELK / Loki / Cloud).
- ✓ Log rotation enabled.
- ✓ Healthchecks monitored.
- ✓ Metrics collected (CPU, Memory, etc.).
- ✓ Alerts configured for critical issues.
- ✓ Application logs are meaningful.

POPULAR LOGGING STACKS



ELK Stack



Grafana



Prometheus

7. ORCHESTRATION & SCALE

- ✓ Docker Compose file tested.
- ✓ Services have restart policies.
- ✓ Healthchecks used for orchestration.
- ✓ Resources (CPU/Memory) defined.
- ✓ Scaling strategy defined.
- ✓ Dependencies and order configured.
- ✓ Use rolling updates for zero downtime.



COMPOSE BEST PRACTICES

- Keep compose.yml clean.
- Use .env files for configs.
- Limit container privileges.

8. PERFORMANCE

- ✓ Images are lightweight.
- ✓ Resource limits set.
- ✓ Unnecessary services disabled.
- ✓ Use caching for builds.
- ✓ Database connections optimized.
- ✓ Use appropriate storage driver.
- ✓ Monitor performance continuously.



PERFORMANCE CHECKS

- Monitor. Measure.
- Optimize. Repeat.
- Performance is continuous.

9. BACKUP & DISASTER RECOVERY

- ✓ Data backup strategy in place.
- ✓ Volumes backed up regularly.
- ✓ Images can be rebuilt easily.
- ✓ Disaster recovery plan documented.
- ✓ Test restore process regularly.



BACKUP STRATEGY

- Backup your data.
- Document your steps.
- Test your recovery.
- Be ready for anything.

10. FINAL PRE-PRODUCTION CHECKS

- ✓ All tests passed.
- ✓ Security scan passed.
- ✓ Performance tested.
- ✓ Rollback plan ready.
- ✓ Documentation updated.
- ✓ Team notified.
- ✓ You are confident and ready to deploy.



PRODUCTION DEPLOYMENT FLOW



Check
Everything



Build
Image



Push to
Registry



Deploy to
Production



Monitor &
Observe



Secure &
Maintain

DONE? GREAT!



You're not just running
containers.
You're running
production systems.

Stay curious.
Keep learning.
Keep shipping.



A checklist today. A stable system tomorrow.
Build it right. Run it right. **Scale** it right.

